EMERGING TECHNOLOGIES | TEAM SPEAKER NOTES

Presentation speaking order

Trần Việt Hải

English narration for assigned slides, in the classroom display order.

PRESENTER	ROLE	SLIDES	COVERAGE
Trần Việt Hải	Opening	**1–4**	Introduction, problem, project goals
Lâm Xê Khoa	Main presenter 1	**5–15**	LLMs, RAG, architecture, source PDFs, metadata
Mai Tuấn Dũng	Main presenter 2	**16–26**	Chunking, embeddings, FAISS, pipelines, LCEL, prompting
Sơn Nguyên	Main presenter 3	**27–37**	Ollama, app.py, logging, evaluation, risks, conclusion
Châu Ngọc Đức	Demo lead	**38**	Observed LangChain question; launch/operate demo
Trần Việt Hải	Demo + closing	**39–40**	Out-of-PDF question; instructor questions; closing

Display order

Slides 1–30 → original slides 33–39 → original slides 31–32 → final Questions slide 40. The presentation-order PowerPoint already reflects this sequence.

Duc operates the Windows 11 terminal and introduces the real LangChain screenshot. Hai demonstrates an out-of-PDF question, enters the instructor's questions, and closes the presentation. Both can assist with the live Q&A.

Demonstration safeguard

Use only observed outputs and verify answers against the sample PDF. A retrieved source list alone does not establish that the generated answer is supported. The old slides 31–32 become slides 38–39 in the speaking-order edition.

SLIDE 01 / 40 • INTRODUCTION • SPEAKER: Trần Việt Hải

Ask a PDF with RAG

WHAT TO SAY

Hello everyone. Today our team presents a PDF question-answering application built with LangChain, FAISS, and Ollama. We will use our actual Windows 11 Python project as the case study, not just describe artificial intelligence in theory. The user can put PDF files into a folder, ask questions in Vietnamese, and view the retrieved source passages. We will also show a case where the system refuses to answer and another case where the model gives an answer that needs verification.

KEY TAKEAWAY

A real RAG prototype—with strengths and limitations we can inspect.

What You Will See Today

WHAT TO SAY

Our presentation moves through five parts. First, we explain why a language model needs access to relevant documents. Second, we show how the RAG pipeline indexes PDFs and retrieves information. Third, we examine functions from our own Python source code. Fourth, we present two actual terminal screenshots. Finally, we evaluate what the examples demonstrate, what they do not prove, and what should be improved.

KEY TAKEAWAY

We connect concepts to code and observed evidence.

SLIDE 03 / 40 • PROBLEM • SPEAKER: Trần Việt Hải

The Problem

WHAT TO SAY

A normal PDF viewer requires users to search, read, and interpret information manually. When documents become longer, this process can be slow. Our project gives users a natural-language interface to one or more PDFs. The application retrieves text that appears related to a question and asks a local language model to produce an answer. This can save time, but the generated answer still needs to be checked against the source.

KEY TAKEAWAY

RAG changes how users access documents—not whether evidence matters.

SLIDE 04 / 40 • PROBLEM • SPEAKER: Trần Việt Hải

Project Goal and Outputs

WHAT TO SAY

The input consists of PDF documents placed in the data/pdfs folder and a question entered in the terminal. The output contains a generated answer, a list of retrieved documents with file names and page numbers, and elapsed time. The application also saves a history file in JSONL format. It is designed as a classroom prototype for text-based PDFs. It does not currently perform OCR on scanned images.

KEY TAKEAWAY

Input: PDFs + question. Output: answer + source metadata + elapsed time.

SPEAKER HANDOFF / LIVE DEMO LINE

Now Khoa will explain the LLM and RAG foundations of our project.

Observed Demo B: Out-of-Scope Question

WHAT TO SAY

This screenshot asks in Vietnamese who the president of the United States is. The question is outside the scope of the bundled book-club PDF. The system returns the configured refusal message, saying it cannot find the information in the PDF. It also displays retrieved chunk metadata and reports 2.10 seconds. This shows one observed example of the refusal behavior, but it does not prove that the system will reject every unsupported question.

KEY TAKEAWAY

The refusal worked in this captured example.

SPEAKER HANDOFF / LIVE DEMO LINE

We are ready for your questions. We can enter a question into the live terminal and review the supporting PDF passages together.

Evidence note: This slide uses a genuine user-supplied terminal screenshot. Do not claim a benchmark or source-grounded accuracy from one screenshot.

SLIDE 40 / 40 • CONCLUSION • SPEAKER: Trần Việt Hải

Questions and References

WHAT TO SAY

Thank you for listening to our presentation. Our main message is that the most useful RAG demonstration includes its architecture, exact implementation, observed behavior, and limitations. The project source, sample PDF, and console screenshots are our primary evidence. For additional technical details, the official LangChain, Ollama, and Hugging Face documentation can be consulted. We welcome questions about the source code, retrieval settings, and how we would test answer faithfulness.

KEY TAKEAWAY

Thank you—questions are welcome.